

# OWASP API Security Top 10

Edicion 2023 — Guia de Estudio en Espanol

Preparacion para Techno Week 8.0 — Banco de Costa Rica

Esta guia cubre las 10 vulnerabilidades mas criticas en APIs segun OWASP (2023). Para cada una incluye: que es, un ejemplo real, un ejemplo aplicado a banca, como explicarlo en 30 segundos (para la charla), y como prevenirlo.

| #     | VULNERABILIDAD                        | EN UNA FRASE                              |
|-------|---------------------------------------|-------------------------------------------|
| API1  | Broken Object Level Authorization     | Un usuario accede a datos de otro         |
| API2  | Broken Authentication                 | Tokens mal implementados o sin expiracion |
| API3  | Broken Object Property Level Auth     | Mass assignment y campos ocultos          |
| API4  | Unrestricted Resource Consumption     | Sin rate limiting, vulnerable a DDoS      |
| API5  | Broken Function Level Authorization   | User normal accede a endpoints de admin   |
| API6  | Unrestricted Access to Business Flows | Abuso de logica de negocio                |
| API7  | Server Side Request Forgery (SSRF)    | La API hace requests a donde no debe      |
| API8  | Security Misconfiguration             | Headers faltantes, CORS permisivo         |
| API9  | Improper Inventory Management         | APIs shadow, versiones viejas activas     |
| API10 | Unsafe Consumption of APIs            | Confiar ciegamente en APIs de terceros    |

## API1:2023 — Broken Object Level Authorization (BOLA)

Un usuario accede a los datos de otro usuario  
[Severidad: CRITICA]

Que es:

Ocurre cuando la API no verifica si el usuario que hace el request tiene permiso para acceder al objeto especifico que esta solicitando. El atacante simplemente cambia el ID en la URL y accede a datos de otro usuario. Es la vulnerabilidad #1 porque es extremadamente comun y facil de explotar: solo hay que cambiar un numero en la URL.

Ejemplo del mundo real:

*Dell (2024): Un atacante creo una cuenta de partner, encontro un endpoint de API que recibia un 'service tag' y retornaba datos del cliente. Genero miles de tags por minuto y extrajo 49 millones de registros. 700Credit*

*(2025): API sin validacion de IDs expuso 5.8 millones de registros financieros.*

**Ejemplo en banca (para tu charla):**

Un cliente del banco consulta su saldo en `/api/accounts/12345/balance`. Cambia el 12345 por 12346 y puede ver el saldo de otro cliente. La API no verifico que la cuenta 12346 perteneciera al usuario autenticado.

**Como explicarlo en 30 segundos:**

*'Imaginen que entran al banco con su cedula, piden su saldo, y el cajero se los da. Pero si dicen el numero de cuenta del vecino, el cajero tambien les da ese saldo sin verificar. Eso es BOLA: la API confia en que el usuario solo va a pedir lo suyo.'*

**Como prevenirlo:**

Implementar verificacion de autorizacion en CADA endpoint, no solo autenticacion. Usar UUIDs en vez de IDs secuenciales (mas dificil de adivinar). En cada request, verificar: el usuario autenticado tiene acceso a ESTE objeto especifico?

**Dato clave:** BOLA representa el 40% de todos los ataques a APIs. Es la #1 desde 2019.

# API2:2023 — Broken Authentication

**Tokens mal implementados, sin expiracion, o mecanismos debiles**

**[Severidad: CRITICA]**

---

Que es:

Ocurre cuando los mecanismos de autenticacion estan mal implementados: tokens JWT que no expiran, passwords debiles permitidos, falta de rate limiting en el login (permitiendo brute force), API keys usadas como autenticacion de usuario, o falta de validacion de la firma del JWT.

Ejemplo del mundo real:

*Multiples breaches han involucrado tokens JWT sin fecha de expiracion o con secretos debiles. Un caso comun: APIs que aceptan tokens firmados con 'none' como algoritmo, permitiendo crear tokens falsos.*

Ejemplo en banca (para tu charla):

La app movil del banco genera un token JWT cuando el cliente se loguea. Pero el token no tiene expiracion. Si un atacante obtiene ese token (por ejemplo, en un WiFi publico), puede usarlo indefinidamente para hacer transferencias en nombre del cliente.

Como explicarlo en 30 segundos:

*'Es como si el banco les diera un gafete de acceso que nunca expira. Si alguien les roba el gafete, puede entrar al banco con su identidad para siempre. Los tokens deben expirar.'*

Como prevenirlo:

Implementar expiracion en todos los JWT (exp claim). Usar secretos fuertes para firmar tokens. Implementar rate limiting en endpoints de login. Nunca usar API keys para autenticar usuarios finales. Validar siempre el algoritmo del JWT (no aceptar 'none').

**Dato clave:** El 52% de incidentes de API en 2026 involucraron autenticacion rota (ThreatStats Report).

# API3:2023 — Broken Object Property Level Authorization

## Mass assignment y exposicion de campos sensibles

[Severidad: ALTA]

---

Que es:

Combina dos problemas: 1) Exposicion excesiva de datos: la API retorna mas campos de los necesarios (por ejemplo, retorna el hash del password junto con el perfil). 2) Mass assignment: el atacante envia campos extra en un request y la API los acepta sin validar (por ejemplo, enviar role=admin en un update de perfil).

Ejemplo del mundo real:

*Un atacante actualiza su perfil enviando {nombre: 'Juan', role: 'admin'} y la API acepta el campo 'role' sin verificar que el usuario no deberia poder cambiarlo. Ahora tiene permisos de administrador.*

Ejemplo en banca (para tu charla):

Un cliente actualiza su direccion de correo en la app del banco. Envia: {email: 'nuevo@mail.com', daily\_transfer\_limit: 99999999}. Si la API no filtra los campos, el cliente acaba de subirse el limite de transferencia diaria a 99 millones.

Como explicarlo en 30 segundos:

*'Es como si llenaran un formulario en el banco para cambiar su direccion, pero en el formulario tambien escribieran 'limite de credito: 100 millones' y el sistema lo procesara sin cuestionar.'*

Como prevenirlo:

Nunca confiar en los campos que envia el cliente. Definir explicitamente que campos son editables. Retornar solo los campos necesarios en cada respuesta (principio de minimo privilegio). En la OpenAPI spec, definir los schemas de request y response por separado.

**Dato clave:** La combinacion de Excessive Data Exposure + Mass Assignment es nueva en 2023 (antes eran #3 y #6 separados).

# API4:2023 — Unrestricted Resource Consumption

**Sin rate limiting, vulnerable a DDoS y abuso**

**[Severidad: ALTA]**

---

Que es:

Ocurre cuando la API no limita la cantidad de requests, el tamaño de los payloads, la cantidad de registros retornados, o el costo de operaciones. Un atacante puede abusar de esto para causar denegación de servicio, generar costos excesivos en cloud, o realizar scraping masivo de datos.

Ejemplo del mundo real:

*Un atacante envía 10,000 requests por segundo al endpoint de búsqueda de una API. Sin rate limiting, el servidor se satura y deja de responder para todos los usuarios.*

Ejemplo en banca (para tu charla):

Un atacante automatiza consultas de saldo para los 20 millones de números de cuenta posibles. Sin rate limiting, en unas horas puede mapear cuáles cuentas existen y cuáles tienen saldo alto. O peor: dispara 100,000 intentos de transferencia por minuto sobrecargando el core bancario.

Como explicarlo en 30 segundos:

*'Imaginen que alguien va a la ventanilla del banco y pide el saldo de 10 mil cuentas diferentes, una por una, sin parar. Sin un límite, el cajero se colapsa. Rate limiting es el guardia que dice: máximo 100 consultas por minuto por usuario.'*

Como prevenirlo:

Implementar rate limiting por usuario, por IP, y por endpoint. Limitar el tamaño de payloads (body size).  
Limitar la cantidad de registros en respuestas paginadas. Implementar timeouts en operaciones costosas.  
Monitorear costos de cloud en tiempo real.

**Dato clave:** Antes se llamaba 'Lack of Resources & Rate Limiting'. El nombre cambio para cubrir mas escenarios.

# API5:2023 — Broken Function Level Authorization

Un usuario normal accede a funciones de administrador

[Severidad: CRITICA]

---

Que es:

Ocurre cuando la API no verifica si el usuario tiene permisos para ejecutar una FUNCION especifica. A diferencia de BOLA (que es sobre objetos/datos), BFLA es sobre acciones: crear, modificar, eliminar. El atacante descubre endpoints administrativos y los ejecuta con su token de usuario normal.

Ejemplo del mundo real:

*Un usuario cambia el metodo HTTP de GET /api/users a DELETE /api/users/123 y la API lo ejecuta porque solo verifica autenticacion pero no autorizacion a nivel de funcion.*

Ejemplo en banca (para tu charla):

Un cliente autenticado en la app del banco descubre el endpoint POST /api/admin/reverse-transaction y puede revertir transferencias que ya se procesaron. O accede a GET /api/admin/all-customers y obtiene la lista completa de clientes del banco.

Como explicarlo en 30 segundos:

*'BOLA es cuando ven los datos de otro. BFLA es cuando pueden HACER cosas que no les corresponden: aprobar prestamos, revertir transacciones, crear cuentas. Es la diferencia entre espiar y actuar.'*

Como prevenirlo:

Implementar control de acceso basado en roles (RBAC). Verificar permisos en CADA endpoint, no solo en la UI. No confiar en que el frontend oculte los botones: si el endpoint existe, alguien lo va a encontrar. Usar middleware de autorizacion que verifique el rol antes de ejecutar la funcion.

**Dato clave:** Los endpoints administrativos son el objetivo mas valioso. Si un atacante los encuentra, game over.

# API6:2023 — Unrestricted Access to Sensitive Business Flows

## Abuso automatizado de logica de negocio

[Severidad: MEDIA]

---

Que es:

Esta vulnerabilidad es nueva en 2023 y es diferente a las demas: no se trata de un bug tecnico sino de abusar de funcionalidades legítimas de forma automatizada. La API funciona correctamente, pero un atacante usa bots para explotar la logica de negocio a escala.

Ejemplo del mundo real:

*Una app de ride-sharing tiene un programa de referidos que da credito por cada amigo invitado. Un atacante automatiza el registro de miles de cuentas falsas, acumulando credits en su cuenta real. La funcionalidad funciona como fue diseñada, pero se abusa de ella.*

Ejemplo en banca (para tu charla):

El banco ofrece una promocion: 'abri una cuenta y recibis 5,000 colones de bienvenida.' Un atacante automatiza la apertura de 10,000 cuentas con datos falsos y acumula 50 millones en bonos de bienvenida. La API de apertura de cuentas funciona bien, pero no tiene proteccion contra abuso.

Como explicarlo en 30 segundos:

*'No es un bug, es un abuso. Es como si alguien entrara 100 veces al supermercado para aprovechar la promocion de una cerveza gratis por visita. Tecnicamente no esta haciendo nada ilegal, pero esta explotando el sistema.'*

Como prevenirlo:

Identificar flujos de negocio sensibles que podrian ser abusados con automatizacion. Implementar CAPTCHA, device fingerprinting, o deteccion de bots en esos flujos. Limitar la frecuencia de operaciones de negocio (no solo requests tecnicos). Monitorear patrones anormales (muchas cuentas desde la misma IP).

**Dato clave:** Esta vulnerabilidad es la mas dificil de detectar con herramientas automaticas porque no es un bug tecnico.

# API7:2023 — Server Side Request Forgery (SSRF)

La API hace requests a destinos que no deberia

[Severidad: ALTA]

---

Que es:

Ocurre cuando la API acepta una URL del usuario y hace un request a esa URL desde el servidor. El atacante puede usar esto para acceder a servicios internos (bases de datos, metadata de cloud, otros microservicios) que no son accesibles desde internet pero si desde el servidor.

Ejemplo del mundo real:

*Capital One (2019): Un atacante exploto una vulnerabilidad SSRF para acceder al servicio de metadata de AWS (169.254.169.254) desde el servidor, obtener credenciales temporales, y acceder a S3 buckets con 100 millones de registros de clientes. Azure API Management (2023): investigadores encontraron 4 vulnerabilidades SSRF que permitian usar servidores de Microsoft como proxies.*

Ejemplo en banca (para tu charla):

La API del banco tiene un endpoint que recibe una URL para verificar un comprobante de pago. Un atacante envia la URL `http://169.254.169.254/latest/meta-data/` y la API, corriendo en AWS, retorna las credenciales del servidor. Con esas credenciales accede a toda la infraestructura.

Como explicarlo en 30 segundos:

*'Es como pedirle a un empleado del banco que llame a un numero de telefono por ustedes. Si el empleado llama a cualquier numero sin verificar, podrian hacerlo llamar a la linea interna del gerente y obtener informacion confidencial.'*

Como prevenirlo:

Nunca hacer requests a URLs proporcionadas por el usuario sin validar. Implementar allowlists de dominios/IPs permitidos. Bloquear acceso a rangos de IP privados (10.x.x.x, 169.254.x.x, 172.16.x.x).  
Deshabilitar el servicio de metadata de cloud cuando no sea necesario (IMDSv2 en AWS).

**Dato clave:** SSRF es la vulnerabilidad que causo el mega-breach de Capital One. Es especialmente peligrosa en cloud.



# API8:2023 — Security Misconfiguration

## Headers faltantes, CORS permisivo, configuraciones por defecto

[Severidad: MEDIA]

---

Que es:

Es la categoria 'catch-all' de errores de configuracion: headers de seguridad faltantes (HSTS, X-Content-Type, CSP), CORS configurado con \* (permitiendo cualquier origen), mensajes de error que exponen stack traces o informacion interna, TLS deshabilitado o mal configurado, permisos por defecto no restringidos, y versiones de software desactualizadas.

Ejemplo del mundo real:

*Una API retorna el stack trace completo cuando hay un error, revelando la version del framework, la estructura de carpetas del servidor, y el nombre de la base de datos. Un atacante usa esa informacion para buscar CVEs conocidos.*

Ejemplo en banca (para tu charla):

La API del banco tiene CORS configurado con Access-Control-Allow-Origin: \* permitiendo que cualquier sitio web haga requests a la API. Un atacante crea un sitio falso del banco, y cuando un cliente lo visita, el sitio puede hacer requests a la API real usando las cookies del cliente.

Como explicarlo en 30 segundos:

*'Es como dejar la puerta del banco abierta de noche porque 'se nos olvido cerrarla'. No es un ataque sofisticado, es un descuido. Y los descuidos son lo que mas explotan los atacantes.'*

Como prevenirlo:

Usar herramientas como Helmet (Node.js) para headers de seguridad. Configurar CORS con dominios especificos, nunca con \*. Deshabilitar stack traces en produccion. Auditar configuraciones regularmente. Usar security headers checkers (securityheaders.com).

**Dato clave:** Es la vulnerabilidad mas facil de prevenir y sin embargo sigue en el #8 desde 2019.

# API9:2023 — Improper Inventory Management

APIs shadow, versiones viejas activas, endpoints olvidados

[Severidad: MEDIA]

---

Que es:

Ocurre cuando la organizacion no tiene un inventario completo y actualizado de todas sus APIs. Esto incluye: versiones viejas de APIs que siguen activas (/api/v1/ junto con /api/v3/), endpoints de desarrollo o testing expuestos en produccion, APIs de terceros integradas sin documentar, y 'shadow APIs' que fueron creadas por equipos sin pasar por el proceso formal.

Ejemplo del mundo real:

*Un banco tiene /api/v1/transfers que ya fue reemplazada por /api/v3/transfers con validaciones de seguridad mejoradas. Pero v1 sigue activa y no tiene las mismas validaciones. Un atacante descubre v1 y la explota.*

Ejemplo en banca (para tu charla):

El equipo de desarrollo creo un endpoint /api/debug/accounts para testing interno. Nunca lo removieron de produccion. Retorna informacion detallada de cualquier cuenta sin autenticacion. Un atacante lo encuentra con un simple scan de la API.

Como explicarlo en 30 segundos:

*'Es como tener una puerta trasera en el banco que se uso durante la construccion y que nunca se cerro. Nadie se acuerda de ella, pero los ladrones si la van a encontrar.'*

Como prevenirlo:

Mantener un inventario centralizado de TODAS las APIs. Deshabilitar versiones viejas activamente. Usar API gateways para controlar que endpoints estan expuestos. Implementar API discovery automatizado para encontrar shadow APIs. Revisar regularmente que no haya endpoints de debug en produccion.

**Dato clave:** Las shadow APIs son el vector de ataque de mas rapido crecimiento. Si no sabes que existe, no la puedes proteger.

# API10:2023 — Unsafe Consumption of APIs

## Confiar ciegamente en las respuestas de APIs de terceros

[Severidad: MEDIA]

---

Que es:

Es nueva en 2023 y se enfoca en un angulo que la mayoría ignora: cuando TU API consume datos de OTRAS APIs (proveedores, partners, servicios de terceros) y confía en esas respuestas sin validarlas. Los atacantes pueden comprometer la API del tercero o interceptar la comunicacion para inyectar datos maliciosos.

Ejemplo del mundo real:

*Una API consume datos de un servicio de geocoding de terceros. El atacante compromete el servicio de geocoding e inyecta payloads de SQL injection en las respuestas. La API confio en los datos del tercero y los paso directo a la base de datos.*

Ejemplo en banca (para tu charla):

El banco usa una API de SINPE para validar cuentas destino antes de transferir. Si la API de SINPE es comprometida y retorna datos manipulados, y el banco confía ciegamente en esa respuesta, las transferencias podrian ir a cuentas del atacante.

Como explicarlo en 30 segundos:

*'Cuando ustedes reciben un paquete de un courier, revisan que sea el correcto antes de abrirlo. Con las APIs de terceros deberia ser igual: validar cada respuesta antes de procesarla. La confianza ciega es una vulnerabilidad.'*

Como prevenirlo:

Validar y sanitizar TODAS las respuestas de APIs de terceros, igual que se valida el input del usuario. Usar TLS para toda comunicacion con APIs externas. Implementar circuit breakers por si una API de terceros es comprometida. No asumir que porque es un 'partner confiable' los datos son seguros.

**Dato clave:** Esta vulnerabilidad se vuelve mas critica con la proliferacion de microservicios: cada servicio confía en el siguiente.

# REFERENCIA RAPIDA PARA LA CHARLA

---

## Si te preguntan 'Cual es la vulnerabilidad mas comun en APIs?'

BOLA (API1). Representa el 40% de ataques a APIs. Es cambiar un ID en la URL y acceder a datos de otro.

## Si te preguntan 'Cual es la mas peligrosa para banca?'

Broken Authentication (API2) + BOLA (API1). Son la combinacion letal: si rompen la autenticacion, BOLA les da acceso a todos los datos.

## Si te preguntan 'Cual es nueva en 2023?'

Unrestricted Access to Sensitive Business Flows (API6) y Unsafe Consumption of APIs (API10). Reflejan amenazas nuevas: abuso de logica de negocio y supply chain.

## Si te preguntan 'Como se relaciona esto con tu pipeline?'

Spectral detecta API1, API2, API4, API8 en la spec. Semgrep detecta API2, API3, API7 en el codigo. ZAP detecta API1, API2, API4, API5, API7, API8 en runtime. Entre los tres cubren 8 de 10. Las que quedan (API6 y API10) requieren logica de negocio que se prueba manualmente.

**Fuente oficial:** <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

**Tiempo de estudio recomendado:** 3-4 horas para esta guia + los ejemplos de OWASP.